

SISTEMAS OPERATIVOS

REPORTE:
ESTRUCTURAS DE PROCESOS

ALUMNO:
JOSÉ PABLO PÉREZ DE ANDA

Profesor:
Josué Padilla Cuevas

Estructura 1

```
/*
 *
 * En este ejemplo, crearé un proceso
 * El cual tendrá 3 hijos en el mismo nivel.
 *
 * Cada hijo imprimirá su PID y el PID de su padre.
 *
 * INDICACIONES:
 * -- Cada proceso debe imprimir su PID y el PID de su padre.
 * -- cada proceso debe de esperar 30 segundos antes de terminar, para
 *    que se pueda ver en consola.
 * */

#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

int main() {

    // Primero crearemos las variables necesarias
    // para almacenar los 3 PID de los hijos.

    pid_t primer_hijo, segundo_hijo, tercer_hijo;

    // remarcaremos el PID del proceso padre
    // para que se vea en consola.
    printf("Soy el proceso padre, este es mi PID: %d y tambien tengo un padre "
        "PPID: %d \n",
        getpid(), getppid());

    // ahora crearemos a cada uno de los hijos utilizando la función fork().
    // por separado para que quede mas claro. Que es lo que esta pasando.

    //[*]
    primer_hijo = fork();
    if (primer_hijo < 0) {

        perror("Error al crear el primer hijo, falta memoria");
        exit(1);

    } else if (primer_hijo == 0) {
        // Estamos en el bloque de memoria del primer hijo.
        // podemos imprimir su PID y EL PPID de su padre.

        printf("HOLA, soy el primer hijo, mi papa PPID es: %d y mi PID es: %d\n",
            getppid(), getpid());
        sleep(45); // Congelamos al proceso 30 segundos
        exit(0); // Salimos del proceso hijo
    }
}
```

```

} else {

    // Estamos dentro del bloque de memoria del proceso padre
    // Supongo que aqui se puede crear al segundo hijo.

    //[*]
    segundo_hijo = fork();

    if (segundo_hijo < 0) {

        perror("Error al crear el segundo hijo, falta memoria");
        exit(1);

    } else if (segundo_hijo == 0) {

        // Estamos en el bloque de memoria del segundo hijo.
        // podemos imprimir su PID y EL PPID de su padre.
        printf("HOLA, soy el segundo hijo, mi papa PPID es: %d y mi PID es: %d\n",
            getppid(), getpid());
        sleep(45); // Congelamos al proceso 45 segundos
        exit(0); // Salimos del proceso hijo

    } else {

        // estamos dentro del bloque de memoria del proceso padre, o eso creo
        // Supongo que aquí se puede crear al tercer hijo.

        //[*]
        tercer_hijo = fork();

        if (tercer_hijo < 0) {

            perror("Error al crear el tercer hijo, falta memoria");
            // exit(1);

        } else if (tercer_hijo == 0) {

            // Estamos en el bloque de memoria del tercer hijo.
            // podemos imprimir su PID y EL PPID de su padre.
            printf(
                "HOLA, soy el tercer hijo, mi papa PPID es: %d y mi PID es: %d\n",
                getppid(), getpid());
            sleep(45); // Congelamos al proceso 45 segundos
            exit(0); // Salimos del proceso hijo

            // podriamos hacer otro hijo :D
        } else {

            // Estamos en el bloque de memoria del proceso padre, o eso creo.
            // No se que mas hacer aqui, creo que ya se han creado los 3 hijos.
            // Solo queda esperar a que terminen los hijos para que el proceso padre
            // termine.
            wait(NULL); // Esperamos a que el primer hijo termine

```

```

    wait(NULL); // Esperamos a que el segundo hijo termine
    wait(NULL); // Esperamos a que el tercer hijo termine
    printf("Todos mis hijos han terminado, BYE ;D\n");
    exit(0); // Salimos del proceso padre
}
}
}

return 0;
}

```

Captura de ejecución

```

Soy el proceso padre, este es mi PID: 7074 y tambien tengo un padre PPID: 7072
HOLA, soy el primer hijo, mi papa PPID es: 7074 y mi PID es: 7075
HOLA, soy el segundo hijo, mi papa PPID es: 7074 y mi PID es: 7076
HOLA, soy el tercer hijo, mi papa PPID es: 7074 y mi PID es: 7077

```

Después de que terminan sleep(45), de manera simultánea se imprime lo de abajo.

```

Todos mis hijos han terminado, BYE ;D

_____ ejecucion finalizada_____

```

captura de htop

PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
7074	pablo	20	0	2468	928	836	S	0.0	0.0	0:00.00	./bin/estructura_uno
7075	pablo	20	0	2468	100	0	S	0.0	0.0	0:00.00	./bin/estructura_uno
7076	pablo	20	0	2468	100	0	S	0.0	0.0	0:00.00	./bin/estructura_uno
7077	pablo	20	0	2468	100	0	S	0.0	0.0	0:00.00	./bin/estructura_uno

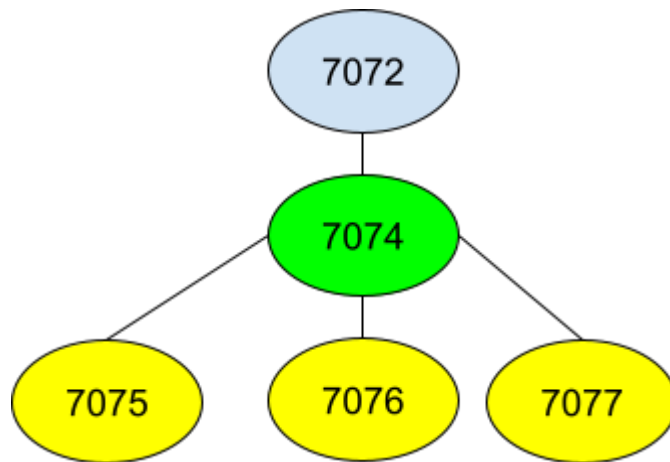
captura pstree -G

```

graph LR
    xfce4-terminal --> bash1[bash]
    xfce4-terminal --> bash2[bash]
    xfce4-terminal --> bash3[bash]
    xfce4-terminal --> bash4[bash]
    bash1 --> htop
    bash2 --> make
    make --> estructura_uno[estructura_uno]
    estructura_uno --> estructuras[3*[estructur+]]
    bash3 --> pstree
    bash4 --> bash5[bash]
    bash5 --> xfce4_terminals[2*[xfce4-terminal]]
    xfce4_terminals --> openbox[2*[openbox]]

```


Dibujito 😊



Estructura 2

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

int main() {

    // Primero crearemos las variables necesarias
    // para almacenar los 3 PID de los hijos.

    pid_t hijo_A, hijo_B;

    printf("Hola soy el PADRE DE TODO ODIN, mi PID es: %d , y mi padre es PPID: %d \n",getpid(),
getppid());

    //[*] ===== PRIMER HIJO =====
    hijo_A = fork();

    if(hijo_A < 0){
        perror("no se pudo crear al proceso hijo_A, T-T");
        exit(1);
    }else if(hijo_A == 0){
        //Estamos en la memoria del hijo_A :O

        //[*HIJO A]
        printf("Hola soy el HIJO A, mi PID es: %d , y mi padre es PPID: %d \n",getpid(), getppid());
        printf("Te adelanto que tendre a 2 hijos, magni y modii\n");
        // Este hijo es prospero y tendra a su vez 2 hijos
```

pid_t nieto_A , nieto_B ; // pienso que estos deben de ser conocidos por el hijo que es el que planea tenerlos

// pero desconosco si aun asi el padre de todo los puede observar

//[procreamos al hijo ===== NIETO_A =====

nieto_A = fork();

if(nieto_A < 0){

printf("soy el hijo A y no pude tener a mi primer hijo, no tengo recursos T-T\n");

exit(1);

}else if(nieto_A == 0){

// Estamos en la memoria del primer hijo de A (magni :D)

printf("Hola soy el nieto_A, mi PID es: %d , y mi padre[hijo_A] es PPID: %d \n",getpid(), getppid());

//[concebimos al bisnietoA]

pid_t bisnietoA;

bisnietoA = fork();

if(bisnietoA < 0){

perror("no se pudo concebir al bisnietoA, no tengo recursos");

exit(1);

}else if (bisnietoA == 0){

//Estamos en la memoria de kratos

printf("Hola soy el bisnieto_A, mi PID es: %d , y mi padre[nieto_A] es PPID: %d \n",getpid(), getppid());

sleep(45);

exit(0);

}else {

// dentro de la memoria de nieto_A

// supongo que aqui es donde debemos de terminar a que espere la ejecucion del buen kratos

sleep(45);

wait(NULL); //esperamos a kratos

printf("el bisnieto_A [Kratos] a terminado su objetivo\n");

exit(0); //-> salimos del proceso padre nieto_A

}

}else {

//regresamos a la memoria del hijo A

// ¿sera que aqui concebimos al bisnieto_B ?

// planeamos tener al chamaco modi

printf("Soy El hijo A [PID: %d], ya tube mi primer hijo (magni), pero ahora \n",getpid());

printf("deseo tener otro al que llamare modi\n");

nieto_B = fork();

if(nieto_B < 0){

printf("lamentablemente como hijo_A, no pude concebir a mi hijo_B [modi]\n");

exit(1);

```

    }else if(nieto_B == 0){
        // estamos en la memoria del nieto_B
        // no planea tener descendencia, simplemente informaremos
        printf("Soy el nieto B, mi [PID]es: %d y mi padre [PPID] es: %d \n", getpid(),getppid());
        sleep(45);
        exit(0);
    }else {
        // regresamos a la memoria del hijo_A
        // aquí debemos de esperar a que su hijo B termine de ejecutar si trabajo

        sleep(45);
        wait(NULL); //esperamos el acta de defunción del nieto_B
        wait(NULL);
        printf("Soy el proceso hijo_A [PID = %d] y mis muchachos han terminado su labor\n",getpid());
        exit(0);
    }
}

}else{
    /*AQUI ESTOS DE REGRESO EN LA MEMORIA DEL PADRE DE TODO ODIN*/

    /* ===== creamos al hijo B =====*/
    hijo_B = fork();
    if(hijo_B < 0){

        printf("No fue posible crear al hijo_B, fatan recutsos para poder hacerlo\n");
        exit(1);

    }else if(hijo_B == 0){
        // estamos dentro de la memoria del hijo_B

        //Este hijo, planea tener 2 chamcos, que seran nietos del padre de todo
        pid_t nieto_Ap , nieto_Bp ;

        printf("SOY EL HIJO_B mi PID es : %d y mi padre es: %d\n",getpid(),getppid());

        //le damos vida a nieto_Ap
        //[*] ===== NIETO_AP =====

        nieto_Ap = fork();

        if(nieto_Ap < 0){

            perror("no es posible crear al proceso nieto_Ap, faltan recursos\n");
            exit(1);

        }else if(nieto_Ap == 0){
            //estamos dentro de la memoria del nieto_Ap
            // El cual planea tener un hijo

```

```

printf("HOLA, soy el nieto_Ap, mi PID es: %d y mi padre [PPID] es: %d\n",getpid(),getppid());

// hijos planeados
pid_t bisnieto_B;

//damos vida al bisnieto_B
bisnieto_B = fork();
if(bisnieto_B < 0){
    perror("No fue posible traer a la vida al bisnieto_B T-T, faltan recursos\n");
    exit(1);
}else if(bisnieto_B == 0){
    //memoria del bisnieto_B
    printf("soy el proceso bisnieto_B, mi PID es : %d y mi padre PPID es:
%d\n",getpid(),getppid());
    sleep(45);
    exit(0);
}else {

    //memoria del nieto_Ap
    //esta esperando activamente el desarrollo de bisnieto_B
    sleep(45);
    wait(NULL);
    printf("soy el proceso nieto_Ap [IDE: %d], e informo que bisnieto_B termino su
función\n",getpid());
    exit(0);
}

}else {
    //memoria del proceso hijo_B

    //el hijo_B desea tener otro hijo
    // [*] ===== NIETO_BP =====

    nieto_Bp = fork();
    if(nieto_Bp < 0){
        perror("error, no se pudo crear al nieto_Bp, faltan recursos\n");
        exit(1);

    }else if (nieto_Bp == 0) {
        //memoria del nieto_Bp
        //No desea tener desendencia
        printf("Soy el proceso nieto_Bp con [PID: %d] y mi padre [PPID] es: %d\n",getpid(),getppid());
        sleep(45);
        exit(0);
    }else{

        // REGRESAMOS A LA MEMORIA DE HIJO_B
        //memoria del proceso hijo_B
        // terminamos de esperar al proceso nieto_Bp & AL PROCESO NIETO_AP

```



```

        sleep(45);
        wait(NULL);
        wait(NULL);
        printf("soy el proceso hijo_B [PID: %d] e informo que mi hijos terminaron sus funciones
\n",getpid());
        exit(0);
    }

}

} else{
    // memoria del proceso_padre ODIN
    // aqui esperamos a que terminen las funciones de el hijo_A y el hijo_B

    sleep(45); // dormimos para no matar rapidamente los procesos
    wait(NULL);
    wait(NULL);
    printf("Soy el padre de todo ODIN, [PID = %d] y mis hijos terminaron sus funciones\n",getpid());
}

}

return 0;
}

```

Captura de ejecución

```

_____Iniciando ejecucion_____
Hola soy el PADRE DE TODO ODIN, mi PID es: 28739 , y mi padre es PPID: 28737
Hola soy el HIJO A, mi PID es: 28740 , y mi padre es PPID: 28739
Te adelanto que tendre a 2 hijos, magni y modii
SOY EL HIJO_B mi PID es : 28741 y mi padre es: 28739
Soy El hijo A [PID: 28740], ya tube mi primer hijo (magni), pero ahora
deseo tener otro al que llamare modi
Hola soy el nieto_A, mi PID es: 28742 , y mi padre[hijo_A] es PPID: 28740
Soy el nieto B, mi [PID]es: 28745 y mi padre [PPID] es: 28740
Soy el proceso nieto_Bp con [PID: 28744] y mi padre [PPID] es: 28741
HOLA, soy el nieto_Ap, mi PID es: 28743 y mi padre [PPID] es: 28741
Hola soy el bisnieto_A, mi PID es: 28746 , y mi padre[nieto_A] es PPID: 28742
soy el proceso bisnieto_B, mi PID es : 28747 y mi padre PPID es: 28743

```

después de la espera

```

soy el proceso nieto_Ap [IDE: 28743], e informo que bisnieto_B termino su función
el bisnieto_A [Kratos] a terminado su objetivo
soy el proceso hijo_B [PID: 28741] e informo que mi hijos terminaron sus funciones
Soy el proceso hijo_A [PID = 28740] y mis muchachos han terminado su labor
Soy el padre de todo ODIN, [PID = 28739] y mis hijos terminaron sus funciones

```

Captura de htop

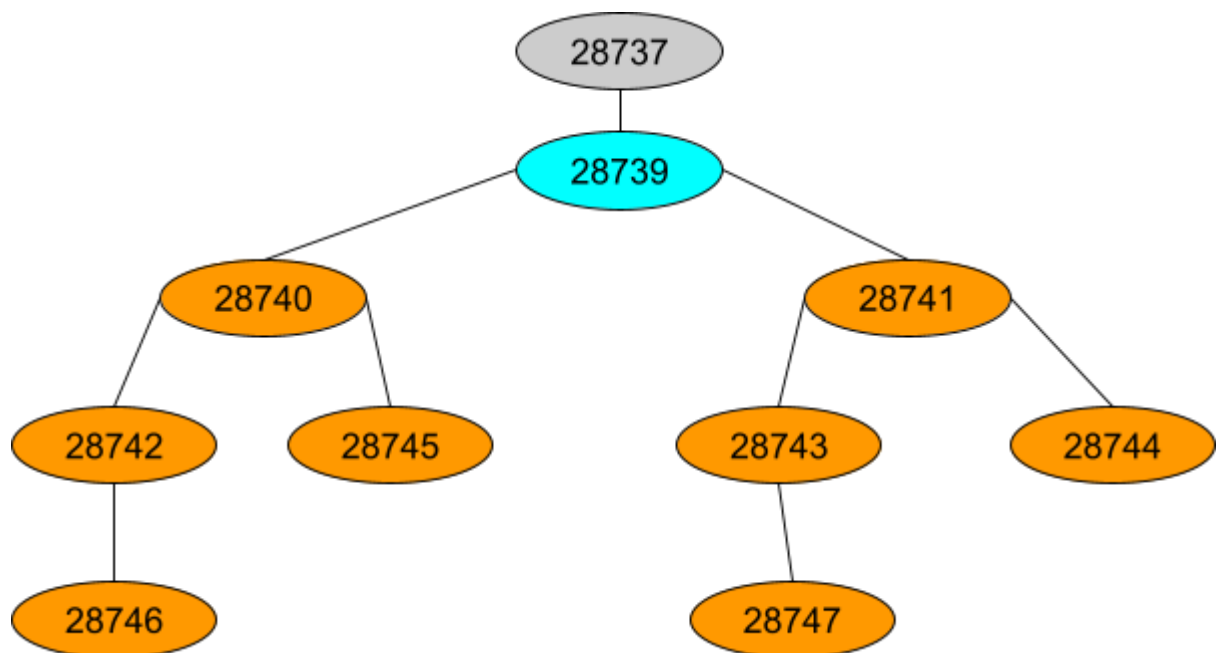
PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
28739	pablo	20	0	2468	872	784	S	0.0	0.0	0:00.00	./bin/estructura_dos
28740	pablo	20	0	2468	96	0	S	0.0	0.0	0:00.00	./bin/estructura_dos
28741	pablo	20	0	2468	96	0	S	0.0	0.0	0:00.00	./bin/estructura_dos
28742	pablo	20	0	2468	96	0	S	0.0	0.0	0:00.00	./bin/estructura_dos
28743	pablo	20	0	2468	96	0	S	0.0	0.0	0:00.00	./bin/estructura_dos
28744	pablo	20	0	2468	96	0	S	0.0	0.0	0:00.00	./bin/estructura_dos
28745	pablo	20	0	2468	96	0	S	0.0	0.0	0:00.00	./bin/estructura_dos
28746	pablo	20	0	2468	96	0	S	0.0	0.0	0:00.00	./bin/estructura_dos
28747	pablo	20	0	2468	96	0	S	0.0	0.0	0:00.00	./bin/estructura_dos

Captura pstree -G

```
graph TD
    xfbrowser["(x-www-browser)(28752)"]
    xfce4term1497["xfce4-terminal(1497)"]
    bash1525["bash(1525)"]
    htop1552["htop(1552)"]
    bash1558["bash(1558)"]
    make28737["make(28737)"]
    estructura_dos28739["estructura_dos(28739)"]
    estructura_dos28740["estructura_dos(28740)"]
    estructura_dos28742["estructura_dos(28742)"]
    estructura_dos28746["estructura_dos(28746)"]
    estructura_dos28741["estructura_dos(28741)"]
    estructura_dos28745["estructura_dos(28745)"]
    estructura_dos28743["estructura_dos(28743)"]
    estructura_dos28747["estructura_dos(28747)"]
    estructura_dos28744["estructura_dos(28744)"]
    bash2161["bash(2161)"]
    pstree28755["pstree(28755)"]
    bash2734["bash(2734)"]
    xfce4term1498["(xfce4-terminal)(1498)"]
    xfce4term1499["(xfce4-terminal)(1499)"]

    xfbrowser --- xfce4term1497
    xfce4term1497 --- bash1525
    xfce4term1497 --- bash1558
    bash1525 --- htop1552
    bash1558 --- make28737
    make28737 --- estructura_dos28739
    estructura_dos28739 --- estructura_dos28740
    estructura_dos28739 --- estructura_dos28741
    estructura_dos28740 --- estructura_dos28742
    estructura_dos28740 --- estructura_dos28745
    estructura_dos28742 --- estructura_dos28746
    estructura_dos28741 --- estructura_dos28743
    estructura_dos28741 --- estructura_dos28744
    estructura_dos28743 --- estructura_dos28747
    bash2161 --- pstree28755
    bash2734 --- xfce4term1498
    bash2734 --- xfce4term1499
```

dibujo



Estructura 3

```
/*
 * Estructura tres
 *
 * bosquejo
 *
 * padreDeTodo[PPID: x , PID:A]
 * |
 * |----HijoA[PPID: A , PID :B]
 * |    |
 * |    |--Nieto_A [PPID: B, PID : C]
 * |    |
 * |    |-- NIETO_B [PPID: B , PID: D]
 * |
 * |----HijoB [PPID: A , PID : E]
 * |
 * |    Nieto_C [PPID:E , PID: F]
 *
 */
```

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
```

```
int main(){
```

```
    // memoria principal
    // PADRE DE TODO
```

```
    // ===== LAS PROMESAS =====
```

```
    pid_t hijo_A, hijo_B ;
```

```
    // ===== identificación =====
```

```
    printf("Soy el padre de todo, mi PID : %d y mi padre PPID es: %d \n",getpid(),getppid());
```

```
    // ===== NACIMIENTO DE HIJO_A =====
```

```
    hijo_A = fork();
```

```
    if( hijo_A < 0){
```

```
        perror("no se pudo crear el proceso hijo_A, faltan recursos");
```

```
        exit(1);
```

```
    }else if(hijo_A == 0){
```

```
        // Estamos en la memoria del HIJO_A
```

```
        printf("Soy el proceso hijo_A, mi PID : %d y mi padre PPID es: %d \n",getpid(),getppid());
```

```
        // ===== PROMESAS =====
```

```

pid_t nieto_A , nieto_B ;

// ===== generamos Nieto_A =====
nieto_A = fork();
if(nieto_A < 0){
    perror("no se puede generar al proceso nieto_A, no hay recursos\n");
    exit(1);
}else if(nieto_A == 0){
    // Estamos en la memoria de nieto_A
    printf("Soy el proceso nieto_A, mi PID : %d y mi padre PPID es: %d \n",getpid(),getppid());
    sleep(45);
    exit(0);
}

//nos dimos cuenta de que el else sobra, ya que si es >=0 implica que
// en este punto estamos en la memoria de A

/*===== generamos nieto_B ===== */
nieto_B = fork();
if(nieto_B < 0){
    perror("no es posible generar al proceso nieto_B, no hay recursos");
    exit(1);
}else if (nieto_B == 0) {
    // Estamos en la memoria nieto_B
    printf("Soy el proceso nieto_B, mi PID : %d y mi padre PPID es: %d \n",getpid(),getppid());
    sleep(45);
    exit(0);
}

// no tiene caso hacer else , ya que si no se esta en nieto_B >= 0
// implica que estaria dentro de la memoria de hijo_A
// aqui es donde esperaremos a los hijos de A, es decir [nieto_A,nieto_B]

// sleep(45); no es necesario, los wait cumplen su mejor la espera
wait(NULL);
wait(NULL);
printf("soy el proceso A, [PID:%d] e informo que mis hijos terminaron su proposito\n",getpid());
exit(0);

}

// aqui tampoco tiene sentido hacer un else
// ya que si HIJO_A != 0 && HIJO_A > 0
// implica que el proceso se encuentra dentro de la memoria del padreDeTodo
// ===== NACIMIENTO DE HIJO_B =====
hijo_B = fork();
if( hijo_B < 0 ){
    perror("no es posible generar el proceso hijo_B, faltan recursos\n");
    exit(1);
}

```

```

}else if( hijo_B == 0 ){
    //ESTAMOS DENTRO DE LA MEMORIA DEL HIJO_B
    printf("Soy el proceso hijo_B, mi PID : %d y mi padre PPID es: %d \n",getpid(),getppid());

    // ===== PROMESAS =====
    pid_t nieto_C;

    // ===== NACIMIENTO de NIETO_C =====
    nieto_C = fork();
    if( nieto_C < 0 ){
        perror("no es posible generar al proceso nieto_C, faltan recursos\n");
        exit(1);
    } else if ( nieto_C == 0 ){
        // ESTAMOS EN LA MEMORIA DE NIETO_C
        printf("Soy el proceso nieto_C, mi PID : %d y mi padre PPID es: %d \n",getpid(),getppid());
        sleep(45);
        exit(0);
    }

    // no tiene sentido hacer el else, ya que si nieto_C != 0 && nieto_C > 0
    // entonces estamos en la memoria de hijo_B
    // ¿que esta haciendo hijo_B en este momento?
    // pues esperando a que su chamaco termine su trabajo

    //sleep(45);
    wait(NULL);
    exit(0);

}

// ¿en este punto que sucede?
// estamos en la memoria del padre todo, al principal
// en principio estamos esperando a que los 2 hijos principales terminen su trabajo
// sleep(45);
wait(NULL);
wait(NULL);
// exit(0) ?
return 0;
}

```

Captura de ejecución

```
Soy el padre de todo, mi PID : 36958 y mi padre PPID es: 36956
Soy el proceso hijo_A, mi PID : 36959 y mi padre PPID es: 36958
Soy el proceso hijo_B, mi PID : 36960 y mi padre PPID es: 36958
Soy el proceso nieto_A, mi PID : 36961 y mi padre PPID es: 36959
Soy el proceso nieto_C, mi PID : 36962 y mi padre PPID es: 36960
Soy el proceso nieto_B, mi PID : 36963 y mi padre PPID es: 36959
```

Captura de htop

PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
36958	pablo	20	0	2468	928	836	S	0.0	0.0	0:00.00	./bin/estructura_tres
36959	pablo	20	0	2468	100	0	S	0.0	0.0	0:00.00	./bin/estructura_tres
36960	pablo	20	0	2468	100	0	S	0.0	0.0	0:00.00	./bin/estructura_tres
36961	pablo	20	0	2468	100	0	S	0.0	0.0	0:00.00	./bin/estructura_tres
36962	pablo	20	0	2468	100	0	S	0.0	0.0	0:00.00	./bin/estructura_tres
36963	pablo	20	0	2468	100	0	S	0.0	0.0	0:00.00	./bin/estructura_tres

captura pstree -p -l

```
xfce4-terminal(1497)
├── bash(1525)──htop(1552)
├── bash(1558)──make(36956)──estructura_tres(36958)──estructura_tres(36959)──estructura_tres(36961)
│                                                    └──estructura_tres(36963)
│                                                    └──estructura_tres(36960)──estructura_tres(36962)
├── bash(2161)──pstree(36971)
├── bash(2734)
├── {xfce4-terminal}(1498)
└── {xfce4-terminal}(1499)
```

dibujo

